

《Bubbles》作业报告

泡泡、种子与海底旅程的小型冒险解谜游戏

1. 程序介绍

《Bubbles》是一款基于 Python 与 Pygame 开发的 2D 冒险解谜游戏。游戏以海底世界为背景，玩家操控一颗承载生命种子的泡泡，在不同关卡中穿过墙体、尖刺、气泡喷口与污染区域，收集自由泡泡和野生种子，最终抵达目标叶片完成关卡。项目强调“泡泡数量”和“种子数量”之间的动态关系：泡泡越多越容易上浮，种子越多越容易下沉，玩家需要通过释放种子、分裂泡泡、收集资源等方式调节浮力，寻找安全路线。

游戏整体风格偏轻量叙事与解谜体验。程序中包含开场剧情、主菜单、关卡地图、暂停菜单、设置、存档、通关结算和结尾场景等内容。关卡设计分为“初生海”和“荆棘礁”两个区域，共 6 个关卡：前几个关卡承担教学功能，引导玩家理解水平移动、释放种子、分裂泡泡、躲避尖刺和利用气泡喷口；后续关卡通过返回叶片、延迟生成物体、资源保留等机制提高策略性。游戏还实现了音效、背景音乐、星级记录、区域解锁、重新开始提示、控制提示开关等体验性功能。

本项目的运行方式较简单。安装依赖后执行 `python main.py` 即可启动游戏。项目使用 `requirements.txt` 管理依赖，核心依赖为 Pygame。源代码采用多目录模块化组织，便于区分游戏主循环、场景逻辑、实体对象、关卡数据、UI 绘制、音频、存档和测试代码。

2. 项目各模块与类设计细节

项目采用“游戏主循环 + 场景切换 + 实体对象 + 数据驱动关卡”的结构。程序入口位于 `main.py`，其中只负责创建 `Game` 对象并调用 `run()` 方法。这样的入口设计保持了启动逻辑的简洁性，也使真正的游戏流程集中在 `core/game.py` 中管理。

2.1 核心控制模块

`Game` 类是整个程序的调度中心。它负责初始化 Pygame、创建窗口、维护时钟、读取事件、调用当前场景的 `handle_events()`、`update()` 和 `draw()` 方法，并根据场景返回的 `action` 切换到主菜单、剧情、关卡、结尾或退出状态。`Game` 类还统一维护音量设置、用户设置和会话进度，避免每个场景重复处理持久化逻辑。

`core/input.py` 将方向键、确认、取消、保存、重开等输入判断封装成函数，降低了不同场景之间的重复代码。`core/sounds.py` 中的 `SoundManager` 封装音效与音乐播放，支持音乐音量和音效音量分开调节。`core/save_manager.py` 中的 `SaveManager` 负责读写 `save_slots.json`，提供 3 个存档位，并保存音量、重开提示、控制提示等设置。存档读写过程中加入了默认值和异常处理，使旧存档或损坏存档不会直接导致程序崩溃。

2.2 场景模块

`scenes/` 目录按游戏流程拆分不同场景。`MenuScene` 管理主菜单、关卡地图、读取存档、设置和区域解锁等界面；`LevelScene` 是实际游玩场景，负责关卡加载、玩家控制、碰撞检测、资源收集、死亡判定、通关判定和结算；`IntroScene` 负责开场故事页；`EndingScene` 负责最终通关后的结束展示。场景之间不直接强耦合，而是通过 `action` 字典向 `Game` 报告“开始关卡”“返回菜单”“进入结尾”等意图，由 `Game` 统一完成切换。

`LevelScene` 内部再组合多个辅助系统：`BubbleMergeSystem` 处理泡泡、种子和融合体的合并规则；`LevelObjectSpawner` 负责按照关卡配置生成自由泡泡、掉落种子和气泡喷口；`LevelStateCodec` 负责把关卡中动态对象序列化为可保存的数据；`SaveFlowMixin` 抽取保存流程，供菜单和关卡结算共同使用。这种组合式设计让 `LevelScene` 虽然承担主要玩法，但不同职责仍能拆开维护。

2.3 实体与物理模块

`entities/player.py` 中的 `Player` 类表示玩家控制的泡泡生命体。它继承自 `FloatBody`，拥有泡泡数、种子数、污染值、半径、碰撞矩形等属性。玩家的核心机制体现在 `net_value = bubble_count - seed_count`：当净值为正时上浮，为负时下沉。`release_seed()` 会减少种子并在玩家下方生成掉落种子，`split_bubble()` 会减少泡泡并在上方生成自由泡泡。若泡泡数过低或污染值达到上限，玩家进入死亡状态。

`core/physics.py` 中的 `FloatBody` 抽象了可浮动对象的通用行为，包括垂直运动、边界限制、与墙体的纵向和横向碰撞修正。实体对象如野生种子、自由泡泡、掉落种子和融合泡泡都可以复用该浮力逻辑。这样既减少重复代码，也保证所有相关对象遵守同一套浮沉规则。

`entities/objects.py` 定义了关卡中的主要对象：`WildSeed` 表示野生种子，`FreeBubble` 表示自由泡泡，`DroppedSeed` 表示玩家释放出的种子，`FusionBubble` 表示泡泡与种子结合后的融合体，`BubbleVent` 表示周期生成自由泡泡的气泡喷口，`Wall` 和 `Spike` 分别表示障碍墙和尖刺，

PollutionZone 表示污染区域，Leaf 表示起点或终点叶片。这些类大多同时包含状态、碰撞区域和绘制方法，便于在 LevelScene 中统一更新和渲染。

2.4 合并、碰撞与关卡机制

BubbleMergeSystem 专门处理玩家与物体、物体与物体之间的合并。玩家碰到野生种子或掉落种子后会增加种子数量，碰到自由泡泡后会增加泡泡数量。若泡泡和种子类对象相互重叠，会生成 FusionBubble。两个融合体相遇时会发生“耗散”，额外溢出一个自由泡泡，这使资源管理不只是简单累加，而具有一定策略成本。系统还负责尖刺刺破泡泡对象、生成破裂特效和清理已收集对象。

关卡数据位于 levels/level_data.py，以列表字典的形式描述关卡名称、区域、起点叶片、目标叶片、玩家出生点、墙体、尖刺、野生种子、自由泡泡、气泡喷口、污染区域和剧情开关。levels/catalog.py 对关卡定义进行字段校验，并提供区域、关卡数量、关卡显示名等查询函数。关卡采用数据驱动后，新增或调整关卡时主要修改配置数据，不需要大量改动玩法代码。

2.5 UI、反馈与测试设计

ui/ 目录负责界面绘制与交互表现，包括菜单视图、地图视图、暂停菜单、结算界面、提示覆盖层、区域解锁动画和通用控件。项目将 UI 绘制从场景逻辑中拆出，减少了单个场景文件继续膨胀的风险。ui/widgets.py 中封装了星级、玻璃质感面板、控制提示和文字换行等通用绘制函数，提高了界面风格的一致性。

项目包含较完整的自动化测试，测试目录覆盖物理碰撞、泡泡喷口、合并系统、存档流程、设置持久化、暂停菜单、关卡地图、剧情场景、音频同步、控制提示和架构约束等方面。测试的存在一方面帮助验证玩法机制，另一方面也避免后续修改 UI 或关卡数据时破坏已有逻辑。

3. 小组成员分工情况

成员	主要负责内容	具体工作
王小愚	游戏主体编程	负责除 UI 以外的主要游戏内容编程，包括核心玩法、物理与碰撞、关卡逻辑、存档、音频和测试等。
金奕晗	UI 制作与问题修复	负责游戏界面制作，并参与 bug 的定位、修改和体验调整。

申云江	项目协调与测试反馈	负责小组调度、整体测试、试玩反馈整理，并提出功能和体验修改建议。
-----	-----------	----------------------------------

4. 项目总结与反思

本项目完成了一个具有完整开始、游玩、保存、结算和结束流程的小型游戏。相较于只实现单一关卡或单一交互，本项目的重点在于把“泡泡与种子的数量关系”发展成持续贯穿关卡的核心机制。玩家不是单纯躲避障碍，而是需要理解浮力变化、资源消耗和关卡路径之间的联系。释放种子会改变自身重量，分裂泡泡会改变上浮能力，收集资源又会影响之后关卡的状态，这些规则共同形成了游戏的解谜基础。

在程序设计方面，项目使用了较清晰的模块划分。`Game` 负责整体流程，`Scene` 负责不同界面状态，实体类负责自身属性和绘制，核心系统负责物理、合并、存档和音频。这样的结构使项目在不断增加菜单、设置、地图、存档和新关卡时仍然能够维护。尤其是关卡数据驱动的设计，让玩法代码和具体关卡内容相对分离，提高了扩展性。

项目开发过程中也暴露出一些不足。首先，部分场景类特别是 `LevelScene` 和 `MenuScene` 承担的功能仍然较多，后续可以继续拆分为更细的控制器或状态机，降低单文件复杂度。其次，美术资源和动画效果虽然已经能支撑完整体验，但仍然偏程序化，若有更充足时间可以加入更统一的角色动画、过场表现和关卡装饰。再次，游戏目前主要是本地单机体验，存档也使用本地 JSON 文件，未来如果要扩展为更正式的发布版本，还需要考虑打包、跨平台兼容、存档迁移和异常提示等问题。

此外，本项目也让我们意识到，游戏开发者与实际游玩者往往并不站在同一视角。开发者在熟悉规则和关卡机制后，容易默认玩家也能快速理解玩法，从而对关卡难度、提示充分性和操作容错产生误判；同时，一些只会在特定操作顺序或特殊关卡状态下出现的问题，也很难仅靠开发者自测及时发现。因此，在游戏制作过程中引入真实游玩者进行试玩和反馈十分必要。试玩者不仅可以验证玩法是否容易理解、难度曲线是否合理，也能更自然地暴露边界情况和隐藏 bug，使后续修改更贴近实际体验。

总体来看，本项目基本达到了课程作业对完整性、可运行性和创新性的要求。它不仅实现了基础的移动、碰撞和界面切换，还围绕“泡泡—种子—浮力”的核心规则进行了关卡设计和系统扩展。通过本项目，我们进一步熟悉了 Pygame 游戏循环、事件处理、面向对象设计、模块拆分、资源管理、存档持久化和自动化测试等内容，也体会到游戏开发中“机制设计”和“代码结构”需要共同推进：只有规则清楚、结构稳定，后续内容扩展和体验打磨才会更加顺畅。